Transaction Management (Chapter 22 – Connelly)

CSE 341
Introduction to Database Systems
Winter 2024
© Royal Military College of Canada

Objectives

- **Function and importance of transactions.**
- **Properties of transactions.**
- **Concurrency Control**
  - **Meaning of serializability.**
  - **How locking can ensure serializability.**
  - **Deadlock and how it can be resolved.**
  - **How timestamping can ensure serializability.**
  - **Optimistic concurrency control.**
  - **Granularity of locking.**

Pearson Education © 2014

Objectives

- **Recovery Control**
  - **Some causes of database failure.**
  - **Purpose of transaction log file.**
  - **Purpose of checkpointing.**
  - **How to recover following database failure.**
- **Alternative models for long duration transactions.**

Pearson Education © 2014

Introduction

- **We previously discussed the functions that a database management system (DBMS) should provide.**
- **Among these are three closely related functions that are intended to ensure that the database is reliable and remains in a consistent state: transaction support, concurrency control services, and recovery services.**

Transaction Support

Transaction Support

Transaction

Action, or series of actions, carried out by user or application, which reads or updates contents of database.

- **Logical unit of work on the database.**
- **Application program is series of transactions with non-database processing in between.**
- **Transforms database from one consistent state to another.**

Pearson Education © 2014

Example Transaction

```
read(staffNo = x, salary)
salary = salary * 1.1
write(staffNo = x, new_salary)
```

(a)

```
delete(staffNo = x)
for all PropertyForRent records, pno
begin
    read(propertyNo = pno, staffNo)
    if (staffNo = x) then
    begin
        staffNo = newStaffNo
        write(propertyNo = pno, staffNo)
    end
end
```

(b)

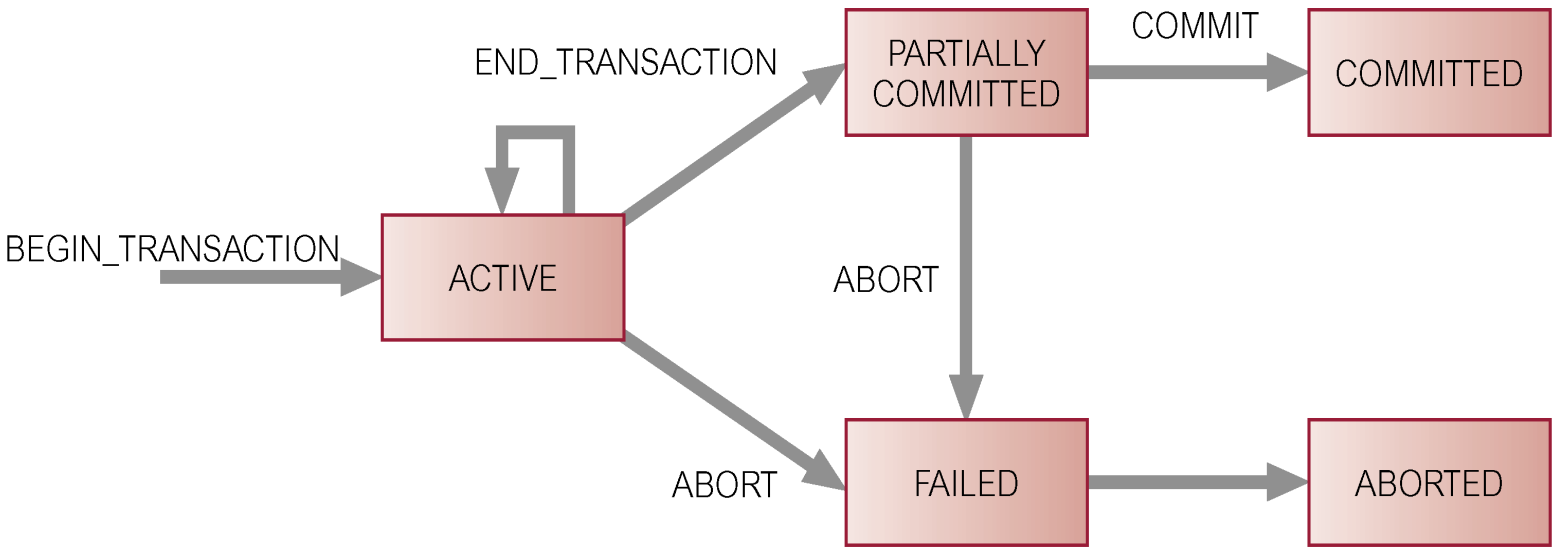
Pearson Education © 2014

Transaction Support

- **Can have one of two outcomes:**
  - **Success - transaction *commits* and database reaches a new consistent state.**
  - **Failure - transaction *aborts*, and database must be restored to consistent state before it started.**
    - **Such a transaction is *rolled back* or *undone*.**
- **Committed transaction cannot be aborted.**
- **Aborted transaction that is rolled back can be restarted later.**

Pearson Education © 2014

State Transition Diagram for Transaction



Pearson Education © 2014

Properties of Transactions

Properties of Transactions

- **Four basic *(ACID)* properties that define a transaction are:**

Atomicity **'All or nothing' property.**

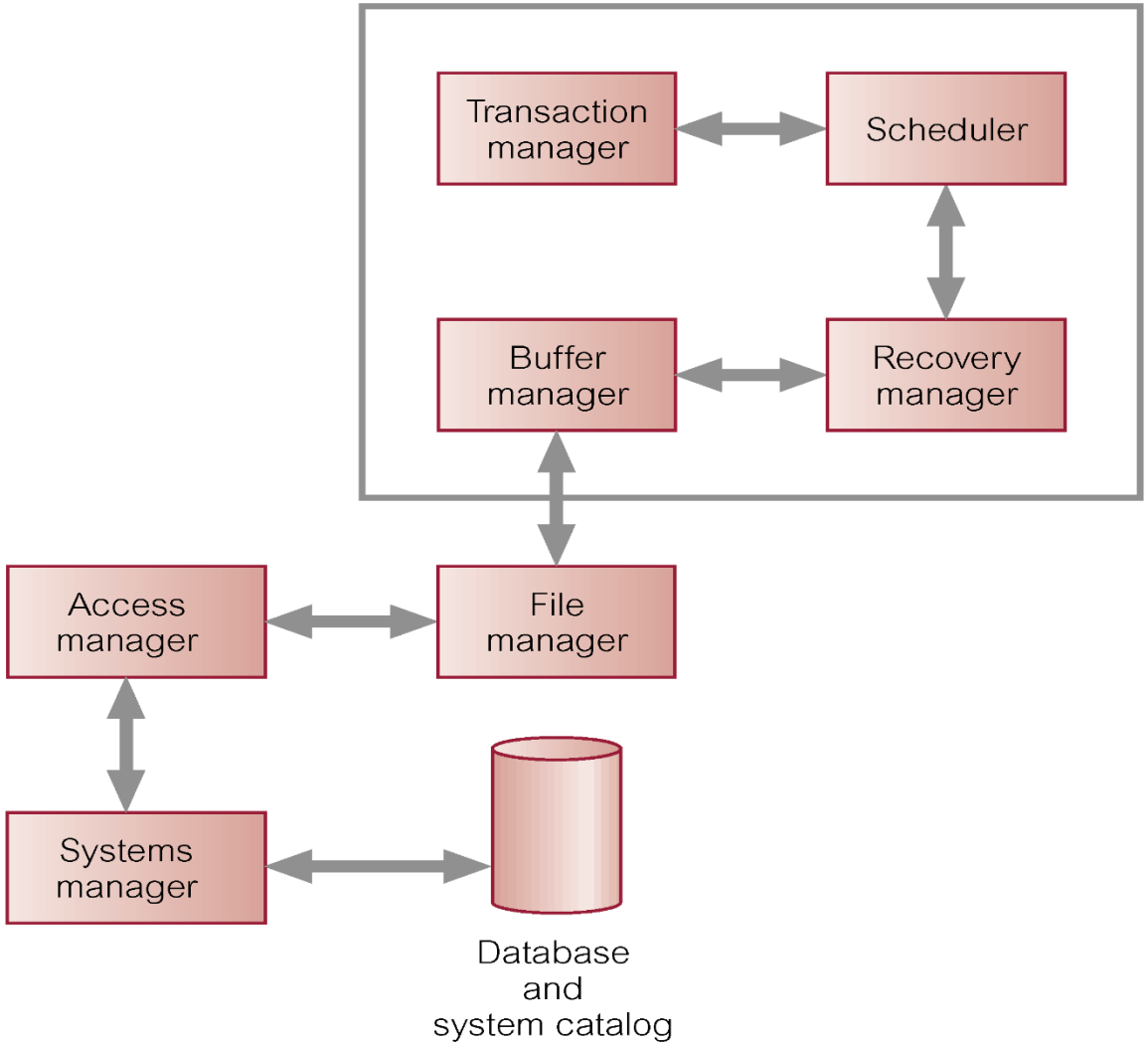
Consistency **Must transform database from one consistent state to another.**

Isolation **Partial effects of incomplete transactions should not be visible to other transactions.**

Durability **Effects of a committed transaction are permanent and must not be lost because of later failure.**

Pearson Education © 2014

DBMS Transaction Subsystem



Pearson Education © 2014

Concurrency Control

Concurrency Control

Process of managing simultaneous operations on the database without having them interfere with one another.

- **Prevents interference when two or more users are accessing database simultaneously and at least one is updating data.**
- **Although two transactions may be correct in themselves, interleaving of operations may produce an incorrect result.**

Pearson Education © 2014

Need for Concurrency Control

Three examples of potential problems caused by concurrency:

1. **Lost update problem.**
2. **Uncommitted dependency problem.**
3. **Inconsistent analysis problem.**

Pearson Education © 2014

Lost Update Problem

- **Successfully completed update is overridden by another user.**
- **T_1 withdrawing £10 from an account with bal_x, initially £100.**
- **T_2 depositing £100 into same account.**
- **Serially, final balance would be £190.**

Pearson Education © 2014

Lost Update Problem

Time	T_1	T_2	bal_x
t_1		begin_transaction	100
t_2	begin_transaction	read(bal_x)	100
t_3	read(bal_x)	$bal_x = bal_x + 100$	100
t_4	$bal_x = bal_x - 10$	write(bal_x)	200
t_5	write(bal_x)	commit	90
t_6	commit		90

- **Loss of T_2's update can be avoided by preventing T_1 from reading bal_x until after update.**

Pearson Education © 2014

Uncommitted Dependency Problem

- **Occurs when one transaction can see intermediate results of another transaction before it has committed.**
- **T_4 updates bal_x to £200 but it aborts, so bal_x should be back at original value of £100.**
- **T_3 has read new value of bal_x (£200) and uses value as basis of £10 reduction, giving a new balance of £190, instead of £90.**

Pearson Education © 2014

Uncommitted Dependency Problem

Time	T_3	T_4	bal_x
t_1		begin_transaction	100
t_2		read(bal_x)	100
t_3		$bal_x = bal_x + 100$	100
t_4	begin_transaction	write(bal_x)	200
t_5	read(bal_x)	⋮	200
t_6	$bal_x = bal_x - 10$	rollback	100
t_7	write(bal_x)		190
t_8	commit		190

- **Problem can be avoided by preventing T_3 from reading bal_x until after T_4 commits or aborts.**

Pearson Education © 2014

Inconsistent Analysis Problem

- **Occurs when transaction reads several values but second transaction updates some of them during execution of first.**
- **Sometimes referred to as *dirty read* or *unrepeatable read*.**
- **T_6 is totaling balances of account x (£100), account y (£50), and account z (£25).**
- **Meantime, T_5 has transferred £10 from bal_x to bal_z, so T_6 now has wrong result (£10 too high).**

Pearson Education © 2014

Inconsistent Analysis Problem

Time	T_5	T_6	bal_x	bal_y	bal_z	sum
t_1		begin_transaction	100	50	25	
t_2	begin_transaction	sum = 0	100	50	25	0
t_3	read(bal_x)	read(bal_x)	100	50	25	0
t_4	$bal_x = bal_x - 10$	sum = sum + bal_x	100	50	25	100
t_5	write(bal_x)	read(bal_y)	90	50	25	100
t_6	read(bal_z)	sum = sum + bal_y	90	50	25	150
t_7	$bal_z = bal_z + 10$		90	50	25	150
t_8	write(bal_z)		90	50	35	150
t_9	commit	read(bal_z)	90	50	35	150
t_{10}		sum = sum + bal_z	90	50	35	185
t_{11}		commit	90	50	35	185

- **Problem avoided by preventing T_6 from reading bal_x and bal_z until after T_5 completed updates.**

Pearson Education © 2014

Serializability

Serializability

- **Objective of a concurrency control protocol is to schedule transactions in such a way as to avoid any interference.**
- **Could run transactions serially, but this limits degree of concurrency or parallelism in system.**
- **Serializability identifies those executions of transactions guaranteed to ensure consistency.**

Pearson Education © 2014

Serializability

<u>Schedule</u>

Sequence of reads/writes by set of concurrent transactions.

<u>Serial Schedule</u>

Schedule where operations of each transaction are executed consecutively without any interleaved operations from other transactions.

- **Serial schedules can differ in their results depending on the order in which transactions are run.**

Pearson Education © 2014

Nonserial Schedule

- **Schedule where operations from set of concurrent transactions are interleaved.**
- **Objective of serializability is to find nonserial schedules that allow transactions to execute concurrently without interfering with one another.**
- **In other words, want to find nonserial schedules that are equivalent to *some* serial schedule. Such a schedule is called *serializable*.**

Pearson Education © 2014

Serializability

- **In serializability, ordering of read/writes is important:**
  - **(a) If two transactions only read a data item, they do not conflict and order is not important.**
  - **(b) If two transactions either read or write separate data items, they do not conflict and order is not important.**
  - **(c) If one transaction writes a data item and another reads or writes same data item, order of execution is important.**

Pearson Education © 2014

Example of Conflict Serializability

Time	T_7	T_8	T_7	T_8	T_7	T_8
t_1	begin_transaction		begin_transaction		begin_transaction	
t_2	read(bal_x)		read(bal_x)		read(bal_x)	
t_3	write(bal_x)		write(bal_x)		write(bal_x)	
t_4		begin_transaction		begin_transaction	read(bal_y)	
t_5		read(bal_x)		read(bal_x)	write(bal_y)	
t_6		write(bal_x)	read(bal_y)		commit	
t_7	read(bal_y)			write(bal_x)		begin_transaction
t_8	write(bal_y)		write(bal_y)			read(bal_x)
t_9	commit		commit			write(bal_x)
t_{10}		read(bal_y)		read(bal_y)		read(bal_y)
t_{11}		write(bal_y)		write(bal_y)		write(bal_y)
t_{12}		commit		commit		commit
	(a)		(b)		(c)	

Equivalent schedules: (a) nonserial schedule S1; (b) nonserial schedule S2 equivalent to S1; (c) serial schedule S3, equivalent to S1 and S2.

This type of serializability is known as conflict serializability. A conflict serializable schedule orders any conflicting operations in the same way as some serial execution.

Pearson Education © 2014

Testing for conflict serializability

- **Conflict serializable schedule orders any conflicting operations in same way as some serial execution.**
- ***Constrained write rule: a transaction updates a data item based on its old value, which is first read by the transaction***
- **Under *constrained write rule*, use *precedence graph* to test for serializability.**

Pearson Education © 2014

Precedence Graph

- **Create:**
  - **node for each transaction;**
  - **a directed edge $T_i \rightarrow T_j$, if T_j reads the value of an item written by T_i;**
  - **a directed edge $T_i \rightarrow T_j$, if T_j writes a value into an item after it has been read by T_i.**
  - **Create a directed edge $T_i \rightarrow T_j$, if T_j writes a value into an item after it has been written by T_i.**

- **If precedence graph contains cycle schedule is not conflict serializable.**

Pearson Education © 2014

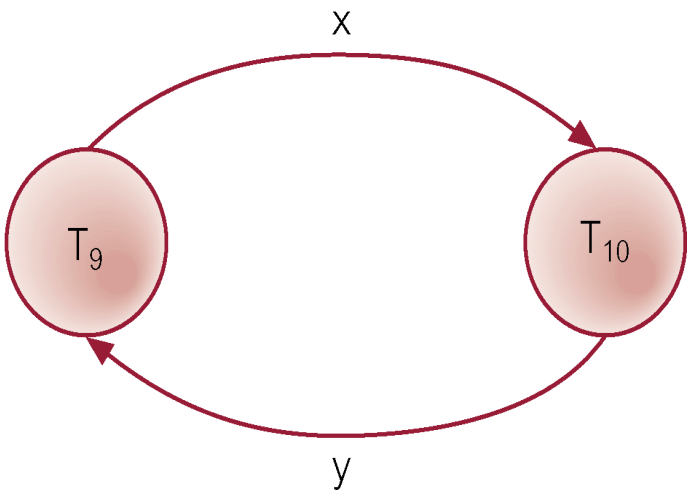
Example - Non-conflict serializable schedule

- **T_9 is transferring £100 from one account with balance bal_x to another account with balance bal_y.**
- **T_{10} is increasing balance of these two accounts by 10%.**
- **Precedence graph has a cycle and so is not serializable.**

Pearson Education © 2014

Example - Non-conflict serializable schedule

Time	T_9	T_{10}
t_1	begin_transaction	
t_2	read(bal_x)	
t_3	$bal_x = bal_x + 100$	
t_4	write(bal_x)	begin_transaction
t_5		read(bal_x)
t_6		$bal_x = bal_x * 1.1$
t_7		write(bal_x)
t_8		read(bal_y)
t_9		$bal_y = bal_y * 1.1$
t_{10}		write(bal_y)
t_{11}	read(bal_y)	commit
t_{12}	$bal_y = bal_y - 100$	
t_{13}	write(bal_y)	
t_{14}	commit	



Pearson Education © 2014

View Serializability

- **View serializability is a more relaxed criterion for the equivalence of schedules compared to conflict serializability.**

Two schedules S_1 and S_2 are view equivalent if:

- For any data item x, if a transaction T_i reads the initial value of x in S_1, then T_i must also read the initial value of x in S_2.
- If a read of x by T_i in S_1 is based on a value written by T_j, then T_i must also read the value of x produced by T_j in S_2.
- If the last write on a data item x is performed by T_i in S_1, then T_i must also perform the final write on x in S_2.

Pearson Education © 2014

View Serializability

- **Schedule is view serializable if it is view equivalent to a serial schedule.**
- **Every conflict serializable schedule is view serializable, although converse is not true.**
- **In general, testing whether schedule is view-serializable is NP-complete.**

Pearson Education © 2014

Example - View Serializable schedule

Time	T_{11}	T_{12}	T_{13}
t_1	begin_transaction		
t_2	read(bal_x)		
t_3		begin_transaction	
t_4		write(bal_x)	
t_5		commit	
t_6	write(bal_x)		
t_7	commit		
t_8			begin_transaction
t_9			write(bal_x)
t_{10}			commit

Blind writes

A schedule is view serializable if the final view of the database, the result of all the reads and writes, is the same as it would be after a serial execution of the transactions.

Pearson Education © 2014

Recoverability

Recoverability

- **Serializability identifies schedules that maintain database consistency, assuming no transaction fails.**
- **Could also examine recoverability of transactions within schedule.**
- **If transaction fails, atomicity requires effects of transaction to be undone.**

Pearson Education © 2014

Recoverable Schedule

A schedule where, for each pair of transactions T_i and T_j, if T_j reads a data item previously written by T_i, then the commit operation of T_i precedes the commit operation of T_j.

Pearson Education © 2014

Concurrency Control Techniques

Concurrency Control Techniques

- **Two basic concurrency control techniques:**
  - **Locking,**
  - **Timestamping.**
- **Both are conservative approaches: delay transactions in case they conflict with other transactions.**
- **Optimistic methods assume conflict is rare and only check for conflicts at commit.**

Pearson Education © 2014

Locking

Transaction uses locks to deny access to other transactions, and so to prevent incorrect updates.

- **Most widely used approach to ensure serializability.**
- **Generally, a transaction must claim a *shared* (*read*) or *exclusive* (*write*) lock on a data item before read or write.**
- **Lock prevents another transaction from modifying item or even reading it, in the case of a write lock.**

Pearson Education © 2014

Locking - Basic Rules

- **If transaction has shared lock on item, can read but not update item.**
- **If transaction has exclusive lock on item, can both read and update item.**
- **Reads cannot conflict, so more than one transaction can hold shared locks simultaneously on same item.**
- **Exclusive lock gives transaction exclusive access to that item.**

Pearson Education © 2014

Locking - Basic Rules

- **Some systems allow transaction to upgrade read lock to an exclusive lock, or downgrade exclusive lock to a shared lock.**

Pearson Education © 2014

Example - Incorrect Locking Schedule

Time	T_9	T_{10}
t_1	begin_transaction	
t_2	read(bal_x)	
t_3	$bal_x = bal_x + 100$	
t_4	write(bal_x)	begin_transaction
t_5		read(bal_x)
t_6		$bal_x = bal_x * 1.1$
t_7		write(bal_x)
t_8		read(bal_y)
t_9		$bal_y = bal_y * 1.1$
t_{10}		write(bal_y)
t_{11}	read(bal_y)	commit
t_{12}	$bal_y = bal_y - 100$	
t_{13}	write(bal_y)	
t_{14}	commit	

For the two transactions, a valid schedule using these rules is:

S = {write_lock(T_9, bal_x), read(T_9, bal_x), write(T_9, bal_x), unlock(T_9, bal_x), write_lock(T_{10}, bal_x), read(T_{10}, bal_x), write(T_{10}, bal_x), unlock(T_{10}, bal_x), write_lock(T_{10}, bal_y), read(T_{10}, bal_y), write(T_{10}, bal_y), unlock(T_{10}, bal_y), commit(T_{10}), write_lock(T_9, bal_y), read(T_9, bal_y), write(T_9, bal_y), unlock(T_9, bal_y), commit(T_9) }

Pearson Education © 2014

Example - Incorrect Locking Schedule

- **If at start, bal_x = 100, bal_y = 400, result should be:**
  - **bal_x = 220, bal_y = 330, if T_9 executes before T_{10}, or**
  - **bal_x = 210, bal_y = 340, if T_{10} executes before T_9.**
- **However, result gives bal_x = 220 & bal_y = 340.**
- **S is not a serializable schedule.**

Pearson Education © 2014

Example - Incorrect Locking Schedule

- **Problem is that transactions release locks too soon, resulting in loss of total isolation and atomicity.**
- **To guarantee serializability, need an additional protocol concerning the positioning of lock and unlock operations in every transaction.**

Pearson Education © 2014

Two-Phase Locking (2PL)

Transaction follows 2PL protocol if all locking operations precede first unlock operation in the transaction.

- **Two phases for transaction:**
  - **Growing phase - acquires all locks but cannot release any locks.**
  - **Shrinking phase - releases locks but cannot acquire any new locks.**
- **Rules:**
  - **A transaction must acquire a lock on an item before operating on the item. The lock may be read or write, depending on the type of access needed.**
  - **Once the transaction releases a lock, it can never acquire any new locks.**

Pearson Education © 2014

Preventing Lost Update Problem using 2PL

Time	T_1	T_2	bal_x
t_1		begin_transaction	100
t_2	begin_transaction	write_lock(bal_x)	100
t_3	write_lock(bal_x)	read(bal_x)	100
t_4	WAIT	$bal_x = bal_x + 100$	100
t_5	WAIT	write(bal_x)	200
t_6	WAIT	commit/unlock(bal_x)	200
t_7	read(bal_x)		200
t_8	$bal_x = bal_x - 10$		200
t_9	write(bal_x)		190
t_{10}	commit/unlock(bal_x)		190

Pearson Education © 2014

Preventing Uncommitted Dependency Problem using 2PL

Time	T_3	T_4	bal_x
t_1		begin_transaction	100
t_2		write_lock(bal_x)	100
t_3		read(bal_x)	100
t_4	begin_transaction	$bal_x = bal_x + 100$	100
t_5	write_lock(bal_x)	write(bal_x)	200
t_6	WAIT	rollback/unlock(bal_x)	100
t_7	read(bal_x)		100
t_8	$bal_x = bal_x - 10$		100
t_9	write(bal_x)		90
t_{10}	commit/unlock(bal_x)		90

Pearson Education © 2014

Preventing Inconsistent Analysis Problem using 2PL

Time	T_5	T_6	bal_x	bal_y	bal_z	sum
t_1		begin_transaction	100	50	25	
t_2	begin_transaction	sum = 0	100	50	25	0
t_3	write_lock(bal_x)		100	50	25	0
t_4	read(bal_x)	read_lock(bal_x)	100	50	25	0
t_5	$bal_x = bal_x - 10$	WAIT	100	50	25	0
t_6	write(bal_x)	WAIT	90	50	25	0
t_7	write_lock(bal_z)	WAIT	90	50	25	0
t_8	read(bal_z)	WAIT	90	50	25	0
t_9	$bal_z = bal_z + 10$	WAIT	90	50	25	0
t_{10}	write(bal_z)	WAIT	90	50	35	0
t_{11}	commit/unlock(bal_x, bal_z)	WAIT	90	50	35	0
t_{12}		read(bal_x)	90	50	35	0
t_{13}		sum = sum + bal_x	90	50	35	90
t_{14}		read_lock(bal_y)	90	50	35	90
t_{15}		read(bal_y)	90	50	35	90
t_{16}		sum = sum + bal_y	90	50	35	140
t_{17}		read_lock(bal_z)	90	50	35	140
t_{18}		read(bal_z)	90	50	35	140
t_{19}		sum = sum + bal_z	90	50	35	175
t_{20}		commit/unlock(bal_x, bal_y, bal_z)	90	50	35	175

Pearson Education © 2014

Cascading Rollback

- **If *every* transaction in a schedule follows 2PL, schedule is serializable.**
- **However, problems can occur with interpretation of when locks can be released.**

Pearson Education © 2014

Cascading Rollback

Time	T_{14}	T_{15}	T_{16}
t_1	begin_transaction		
t_2	write_lock($\mathbf{bal_x}$)		
t_3	read($\mathbf{bal_x}$)		
t_4	read_lock($\mathbf{bal_y}$)		
t_5	read($\mathbf{bal_y}$)		
t_6	$\mathbf{bal_x} = \mathbf{bal_y} + \mathbf{bal_x}$		
t_7	write($\mathbf{bal_x}$)		
t_8	unlock($\mathbf{bal_x}$)	begin_transaction	
t_9	⋮	write_lock($\mathbf{bal_x}$)	
t_{10}	⋮	read($\mathbf{bal_x}$)	
t_{11}	⋮	$\mathbf{bal_x} = \mathbf{bal_x} + 100$	
t_{12}	⋮	write($\mathbf{bal_x}$)	
t_{13}	⋮	unlock($\mathbf{bal_x}$)	
t_{14}	⋮	⋮	
t_{15}	rollback	⋮	
t_{16}		⋮	begin_transaction
t_{17}		⋮	read_lock($\mathbf{bal_x}$)
t_{18}		rollback	⋮
t_{19}			rollback

Pearson Education © 2014

Cascading Rollback

- **Transactions conform to 2PL.**
- **T_{14} aborts.**
- **Since T_{15} is dependent on T_{14}, T_{15} must also be rolled back. Since T_{16} is dependent on T_{15}, it too must be rolled back.**
- **This is called *cascading rollback*.**
- **To prevent this with 2PL, leave release of *all* locks until end of transaction.**

Pearson Education © 2014

Deadlock

An impasse that may result when two (or more) transactions are each waiting for locks held by the other to be released.

Time	T_{17}	T_{18}
t_1	begin_transaction	
t_2	write_lock($\mathbf{bal_x}$)	begin_transaction
t_3	read($\mathbf{bal_x}$)	write_lock($\mathbf{bal_y}$)
t_4	$\mathbf{bal_x} = \mathbf{bal_x} - 10$	read($\mathbf{bal_y}$)
t_5	write($\mathbf{bal_x}$)	$\mathbf{bal_y} = \mathbf{bal_y} + 100$
t_6	write_lock($\mathbf{bal_y}$)	write($\mathbf{bal_y}$)
t_7	WAIT	write_lock($\mathbf{bal_x}$)
t_8	WAIT	WAIT
t_9	WAIT	WAIT
t_{10}	⋮	WAIT
t_{11}	⋮	⋮

Pearson Education © 2014

Deadlock

- **Only one way to break deadlock: abort one or more of the transactions.**
- **Deadlock should be transparent to user, so DBMS should restart transaction(s).**
- **However, in practice DBMS cannot restart aborted transaction since it is unaware of transaction logic even if it was aware of the transaction history.**

Pearson Education © 2014

Deadlock

- **Three general techniques for handling deadlock:**
  - **Timeouts.**
  - **Deadlock prevention.**
  - **Deadlock detection and recovery.**

Pearson Education © 2014

Timeouts

- **Transaction that requests lock will only wait for a system-defined period of time.**
- **If lock has not been granted within this period, lock request times out.**
- **In this case, DBMS assumes transaction may be deadlocked, even though it may not be, and it aborts and automatically restarts the transaction.**

Pearson Education © 2014

Deadlock Prevention

- **DBMS looks ahead to see if transaction would cause deadlock and never allows deadlock to occur.**
- **Could order transactions using transaction timestamps:**
  - **<u>Wait-Die</u> - only an older transaction can wait for younger one, otherwise transaction is aborted (*dies*) and restarted with same timestamp.**
  - **<u>Wound-Wait</u> - only a younger transaction can wait for an older one. If older transaction requests lock held by younger one, younger one is aborted (*wounded*).**

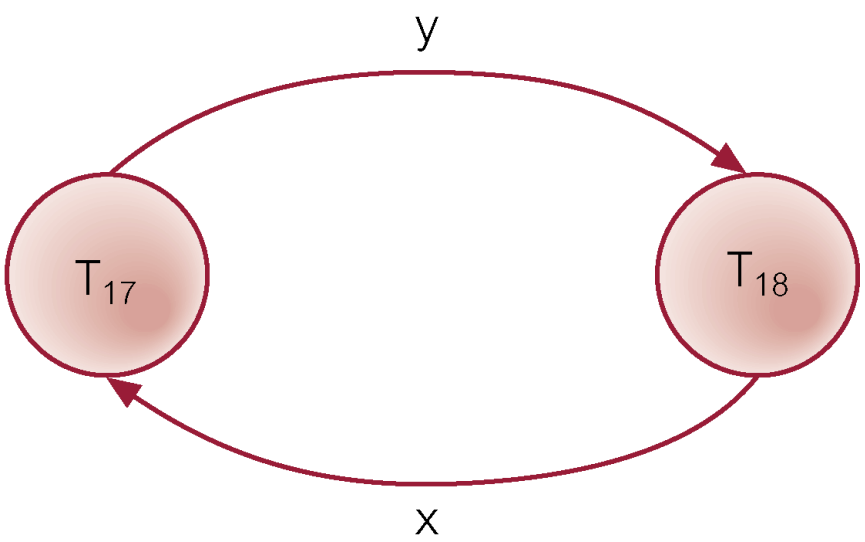
Pearson Education © 2014

Deadlock Detection and Recovery

- **DBMS allows deadlock to occur but recognizes it and breaks it.**
- **Usually handled by construction of wait-for graph (WFG) showing transaction dependencies:**
  - **Create a node for each transaction.**
  - **Create edge T_i -> T_j, if T_i waiting to lock item locked by T_j.**
- **Deadlock exists if and only if WFG contains cycle.**
- **WFG is created at regular intervals.**

Pearson Education © 2014

Example - Wait-For-Graph (WFG)



Pearson Education © 2014

Recovery from Deadlock Detection

- **Several issues:**
  - **choice of deadlock victim;**
  - **how far to roll a transaction back;**
  - **avoiding starvation.**

Pearson Education © 2014

Timestamping

- **Transactions ordered globally so that older transactions, transactions with *smaller* timestamps, get priority in the event of conflict.**
- **Conflict is resolved by rolling back and restarting transaction.**
- **No locks so no deadlock.**

Pearson Education © 2014

Timestamping

<u>Timestamp</u>

A unique identifier created by DBMS that indicates relative starting time of a transaction.

- **Can be generated by using system clock at time transaction started, or by incrementing a logical counter every time a new transaction starts.**

Pearson Education © 2014

Timestamping

- **Read/write proceeds only if *last update on that data item* was carried out by an older transaction.**
- **Otherwise, transaction requesting read/write is restarted and given a new timestamp.**
- **Also timestamps for data items:**
  - **<u>read-timestamp</u> - timestamp of last transaction to read item;**
  - **<u>write-timestamp</u> - timestamp of last transaction to write item.**

Pearson Education © 2014

Timestamping - Read(x)

- **Consider a transaction T with timestamp ts(T) that issues a read(x):**

<u>ts(T) < write_timestamp(x)</u>

- **x already updated by younger (later) transaction.**
- **Transaction must be aborted and restarted with a new timestamp.**

Pearson Education © 2014

Timestamping - Write(x)

Consider a transaction T with timestamp ts(T) that issues a write(x):

<u>ts(T) < read_timestamp(x)</u>

- **x already read by younger transaction.**
- **Roll back transaction and restart it using a later timestamp.**

<u>ts(T) < write_timestamp(x)</u>

- **x already written by younger transaction.**
- **Write can safely be ignored - *ignore obsolete write* rule.**

- **Otherwise, operation is accepted and executed.**

Pearson Education © 2014

Example – Basic Timestamp Ordering

Time	Op	T_{19}	T_{20}	T_{21}
t_1		begin_transaction		
t_2	read(bal_x)	read(bal_x)		
t_3	$bal_x = bal_x + 10$	$bal_x = bal_x + 10$		
t_4	write(bal_x)	write(bal_x)	begin_transaction	
t_5	read(bal_y)		read(bal_y)	
t_6	$bal_y = bal_y + 20$		$bal_y = bal_y + 20$	begin_transaction
t_7	read(bal_y)			read(bal_y)
t_8	write(bal_y)		write(bal_y)[+]	
t_9	$bal_y = bal_y + 30$			$bal_y = bal_y + 30$
t_{10}	write(bal_y)			write(bal_y)
t_{11}	$bal_z = 100$			$bal_z = 100$
t_{12}	write(bal_z)			write(bal_z)
t_{13}	$bal_z = 50$	$bal_z = 50$		commit
t_{14}	write(bal_z)	write(bal_z)[‡]	begin_transaction	
t_{15}	read(bal_y)	commit	read(bal_y)	
t_{16}	$bal_y = bal_y + 20$		$bal_y = bal_y + 20$	
t_{17}	write(bal_y)		write(bal_y)	
t_{18}			commit	

[+] At time t_8, the write by transaction T_{20} violates the first timestamping write rule described above and therefore is aborted and restarted at time t_{14}.

[‡] At time t_{14}, the write by transaction T_{19} can safely be ignored using the ignore obsolete write rule, as it would have been overwritten by the write of transaction T_{21} at time t_{12}.

Pearson Education © 2014